Explicación del código – Proyecto de herramientas de mecánica con Express y MySQL

Samuel Santiago López

Análisis y Desarrollo de Software

Backend: C3312932

Yoly Alexandra Guevara Muñoz

6 de noviembre del 2026

Introducción

En este documento explico con mis palabras cómo funciona el proyecto que hice para guardar y consultar las herramientas del taller de mecánica. Es una API, es decir, un programa que recibe pedidos desde otra aplicación (por ejemplo Postman) y responde con información. Para hacerlo usé Node.js con Express para el servidor, y MySQL como base de datos donde queda guardada la información.

1. ¿Qué hace mi proyecto?

Mi proyecto permite hacer dos cosas con las herramientas del taller: **registrar** una herramienta nueva (con su nombre, la repisa donde está, su código, el área y el instructor encargado) y **ver** todas las herramientas que ya están guardadas.

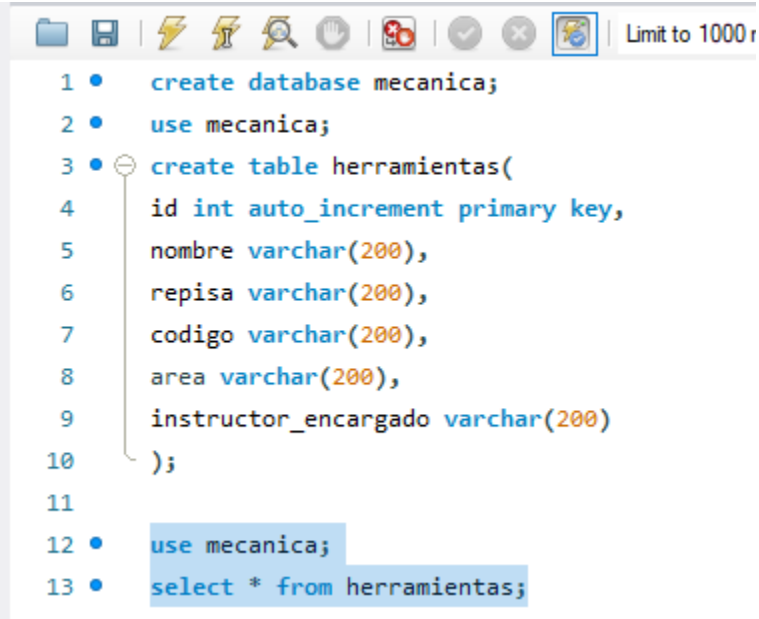
El camino que sigue la información es este: primero alguien envía un pedido desde Postman o desde una página web; el servidor lo recibe, habla con la base de datos MySQL para guardar o buscar los datos, y luego devuelve una respuesta en formato JSON (un formato de texto fácil de leer para los programas).

Los archivos del proyecto son:

- **mecanic.sql:** crea la base de datos y la tabla donde se guardan las herramientas.
- **mecanica.js:** es el servidor. Aquí está todo lo que hace funcionar la API.
- **package.json:** es como la ficha del proyecto: dice cómo se llama, cómo se enciende y qué programas extra necesita.
- **package-lock.json:** guarda las versiones exactas de esos programas extra para que el proyecto funcione igual en cualquier computador.

2. La base de datos en MySQL

Antes de encender el servidor hay que crear el lugar donde se van a guardar los datos. Eso lo hice con este archivo en MySQL Workbench:



- **create database mecánica:** crea la base de datos, que es como una carpeta grande donde va a vivir toda la información del taller.
- **use mecánica :** le dice a MySQL que trabaje dentro de esa base de datos.
- **create table herramientas:** crea la tabla, que funciona como una hoja de Excel: cada herramienta es una fila y cada dato es una columna.
- **select * from herramientas:** muestra todo lo que hay guardado en la tabla. Me sirve para comprobar que la API sí está guardando los datos.

Las columnas de la tabla

- **id:** un número que MySQL asigna solo a cada herramienta (1, 2, 3...). Nunca se repite y sirve para identificarla.
- **nombre:** cómo se llama la herramienta.
- **repisa:** el lugar del taller donde está guardada.
- **codigo:** el código con el que se identifica en el inventario.
- **area:** la zona del taller a la que pertenece.
- **instructor_encargado:** el instructor responsable de esa herramienta.

```
11
12 • use mecanica;
13 • select * from herramientas;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	nombre	repisa	codigo	area	instructor_encargado
1	llave L	A8	f2-993	carros	mario
2	llave L	A8	f2-993	carros	mario
NULL	NULL	NULL	NULL	NULL	NULL

3. El archivo de configuración

El archivo **package.json** es la ficha del proyecto.

package.json

```
{
  "name": "mecanic",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  Debug
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node mecanica.js",
    "dev": "nodemon mecanica.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "cors": "^2.8.6",
    "express": "^5.3.0",
    "mysql2": "^3.24.6"
  },
  "devDependencies": {
    "nodemon": "^3.1.14"
  }
}
```

Las formas de encender el servidor

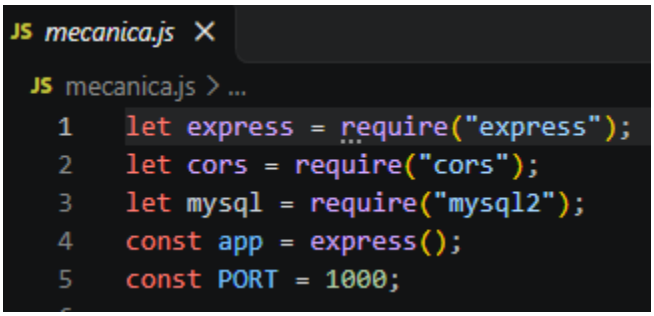
- **npm start** enciende el servidor normalmente.
- **npm run dev** lo enciende con nodemon, que lo reinicia solo cada vez que guardo un cambio en el código. Es lo que uso mientras estoy programando.

Los programas extra que usa el proyecto

- **express:** es la herramienta principal. Con ella creo el servidor y le explico qué hacer cuando llega un pedido.
- **mysql2:** permite que el servidor se comunique con MySQL para guardar y buscar datos.
- **cors:** deja que otras aplicaciones, como una página hecha en Angular, puedan pedirle datos a mi servidor sin que el navegador las bloquee.
- **nodemon:** reinicia el servidor automáticamente. Solo se usa mientras se programa.

4. El servidor paso a paso

Ahora explico el archivo **mecanica.js** en el mismo orden en que está escrito.



4.1 Traer las herramientas y preparar el servidor

Las tres primeras líneas traen los programas que instalé (express, cors y mysql2) para poder usarlos. Luego **app = express()** crea el servidor y **PORT = 1000** define la puerta o el vínculo por donde va a recibir los pedidos. Por eso mi API se abre en **http://localhost:1000**.

4.2 Dos ayudantes del servidor

```
app.use(express.json());
app.use(cors());
```

- **express.json()** hace que el servidor entienda los datos que llegan en formato JSON. Si no lo pongo, los datos que envío desde Postman no se pueden leer y llegan vacíos.
- **cors()** permite que otras aplicaciones puedan comunicarse con mi servidor.

4.3 Conectarse a la base de datos

```
// Conexión a base de datos explicada por la profe :3
let conectar = mysql.createConnection({
    host: "127.0.0.1",
    user: "root",
    password: "Almendra",
    database: "mecanica",
    port: 3306
});

//codigo para saber estado de la conexion
conectar.connect((error) => {
    if (error) {
        console.log("no se a conectado adecuadamente");
    } else {
        console.log("la conexion ha sido exitosa");
    }
});
```

Aquí le doy al servidor los datos que necesita para entrar a MySQL.

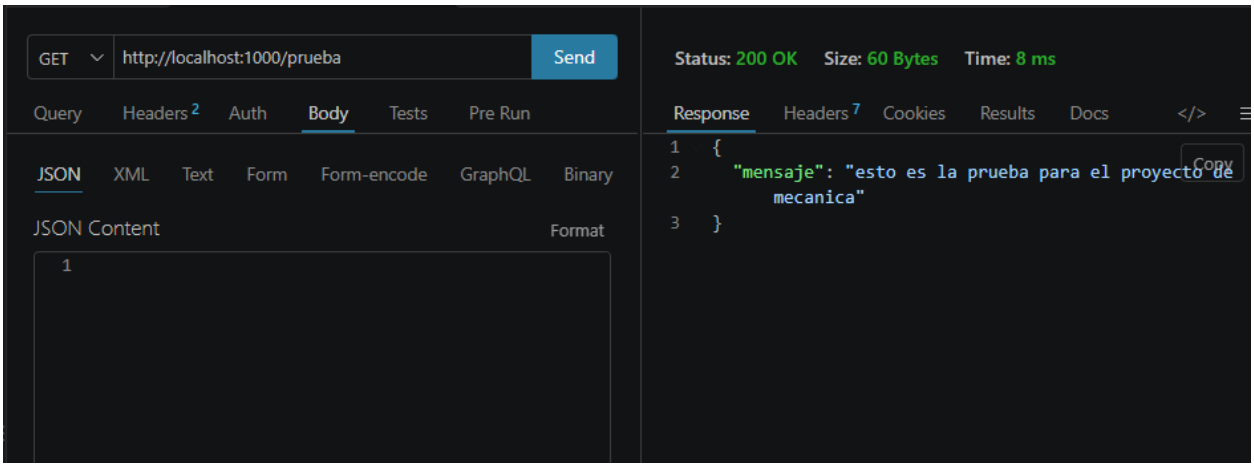
- **host:** dónde está MySQL. **127.0.0.1** significa “en este mismo computador”.
- **user y password:** el usuario y la contraseña para entrar.
- **database:** cuál base de datos usar, en este caso **mecanica**.
- **port:** la puerta de MySQL, que normalmente es la 3306.

Después, **conectar.connect** intenta entrar. Si algo sale mal, escribe en la consola que no se pudo conectar; si todo va bien, escribe que la conexión fue exitosa. Así, apenas enciendo el servidor, sé si la conexión funciona.

4.4 La ruta de prueba

```
// prueba y registro de herramientas
app.get("/prueba", (req, res) => {
    res.json({ "mensaje": "esto es la prueba para el proyecto de mecanica" });
});
```

Esta ruta solo sirve para comprobar que el servidor está encendido. Cuando entro a **http://localhost:1000/prueba**, me responde con un mensaje. No usa la base de datos. En el código, **req** es el pedido que llega y **res** es la respuesta que el servidor devuelve.



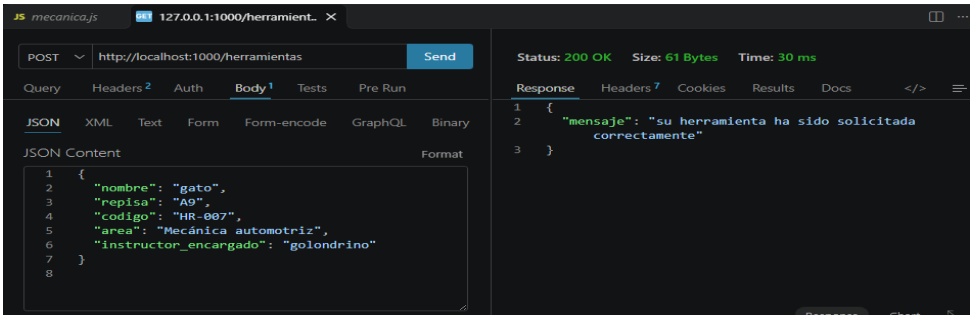
4.5 Registrar una herramienta

```
app.post("/herramientas",(req,res)=>{
    const {nombre,repisa,codigo,area,instructor_encargado}=req.body
    conectar.query("INSERT INTO herramientas (nombre,repisa,codigo,area,instructor_encargado) VALUES
        [nombre, repisa,codigo,area,instructor_encargado],(error,resultado)=>{
            if(error){
                return res.json({"mensaje":"error al pedir prestamo de herramienta"})
            }
            else{
                return res.json({"mensaje":"su herramienta ha sido solicitada correctamente"})
            }
        })
})
```

Esta ruta recibe los datos de una herramienta nueva y la guarda. Funciona así:

1. Saca del pedido los cinco datos: nombre, repisa, código, área e instructor encargado.
2. Le pide a MySQL que guarde una fila nueva con esos datos. Los signos de interrogación? son espacios que se llenan con los datos del arreglo.
3. Cuando MySQL termina, se ejecuta la función que está al final. Si hubo un error, responde con un mensaje de error; si todo salió bien, responde que la herramienta quedó registrada.

Por ejemplo, si desde Postman envío esto:



se crea una fila nueva en la tabla y el servidor responde que la herramienta fue solicitada correctamente.

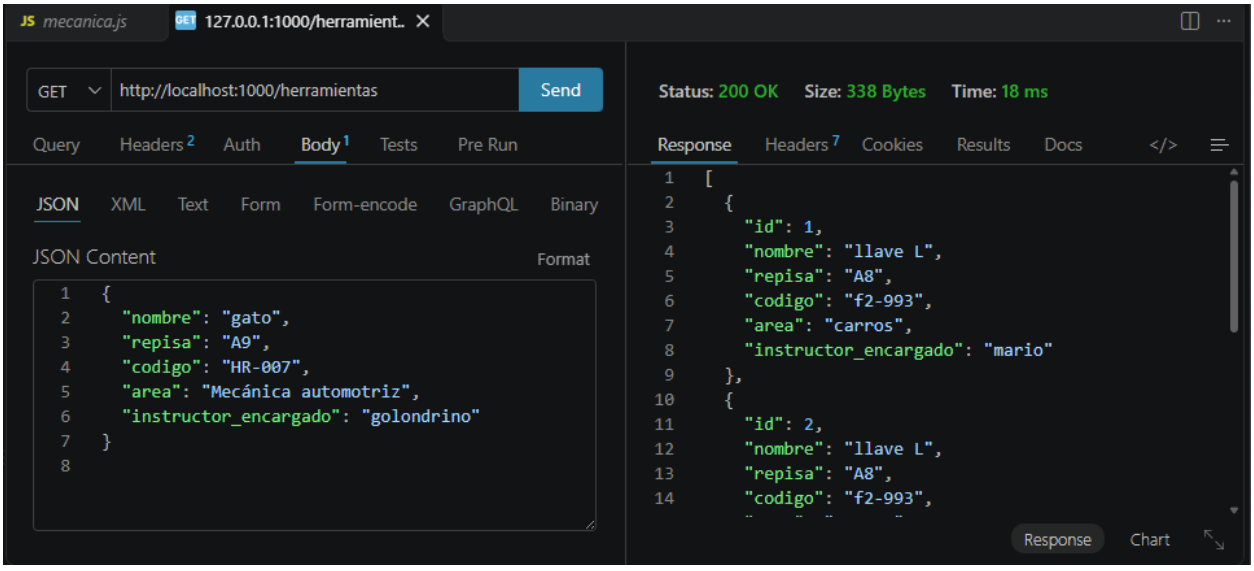
	id	nombre	repisa	codigo	area	instructor_encargado
▶	1	llave L	A8	f2-993	carros	mario
	2	llave L	A8	f2-993	carros	mario
	3	gato	A9	HR-007	Mecá...	golondrino
*	NULL	NU	NULL	NULL	NULL	NULL

gato

4.6 Ver las herramientas guardadas

```
app.get("/herramientas",(req,res)=>{
    conectar.query("SELECT * FROM herramientas",(error,resultados)=>{
        if(error){
            return res.json({mensaje:"error al traer datos de las herramientas"})
        }
        else{
            return res.json(resultados)
        }
    })
})
```

Esta ruta le pide a MySQL todas las herramientas guardadas (**SELECT * FROM herramientas** significa “tráeme todo de la tabla herramientas”) y las devuelve en una lista. Si algo falla, avisa con un mensaje en lugar de quedarse sin responder.



4.7 Encender el servidor

```
app.listen(PORT, () => {
    console.log("servidor en ejecucion" + PORT)
})
```

app.listen deja el servidor escuchando pedidos en el puerto 1000 y escribe un mensaje en la consola.

5. Las rutas de la API

Una ruta es una dirección a la que se le puede hacer un pedido. En mi proyecto hay tres:

- **GET /prueba:** comprueba que el servidor está funcionando. Devuelve un mensaje de confirmación.
- **POST /herramientas:** registra una herramienta nueva en la base de datos. Devuelve un mensaje diciendo si se pudo guardar o no.
- **GET /herramientas:** muestra la lista de todas las herramientas guardadas.

La diferencia entre las dos primeras palabras es simple: **GET** sirve para pedir información sin cambiar nada, y **POST** sirve para enviar información nueva que se va a guardar.

Conclusión

Con este trabajo entendí cómo se conectan las partes de una aplicación: quien hace el pedido, el servidor que lo recibe y la base de datos donde queda guardada la información. Cada línea del código tiene un papel: unas preparan el servidor, otras lo conectan con MySQL y otras definen qué pasa cuando llega un pedido para registrar o consultar herramientas.